Machine Learning Group

ULB

# Digital Twins with NVIDIA Isaac Sim & Isaac Lab

# Agenda

- Isaac Sim / Gym / Lab: introduction, architecture, and technical requirements
- Comparison with direct competitors
- Advantages and limitations in the context of our projects (examples for robotics and virtual power plants)
- Conclusions

# Isaac **Sim**, Isaac **Gym**, Isaac **Lab**

- **Isaac Sim** (proprietary core, open APIs): high-fidelity virtual robotics lab, PhysX 5, RTX photoreal rendering, rich sensors (RGB-D, LiDAR, IMU), ROS/ROS2 bridges, Replicator for synthetic data, huge USD asset ecosystem.

  Example of industrial use: **BMW** uses Isaac Sim to **virtualize factory planning** and validate robotics workflows before deployment.

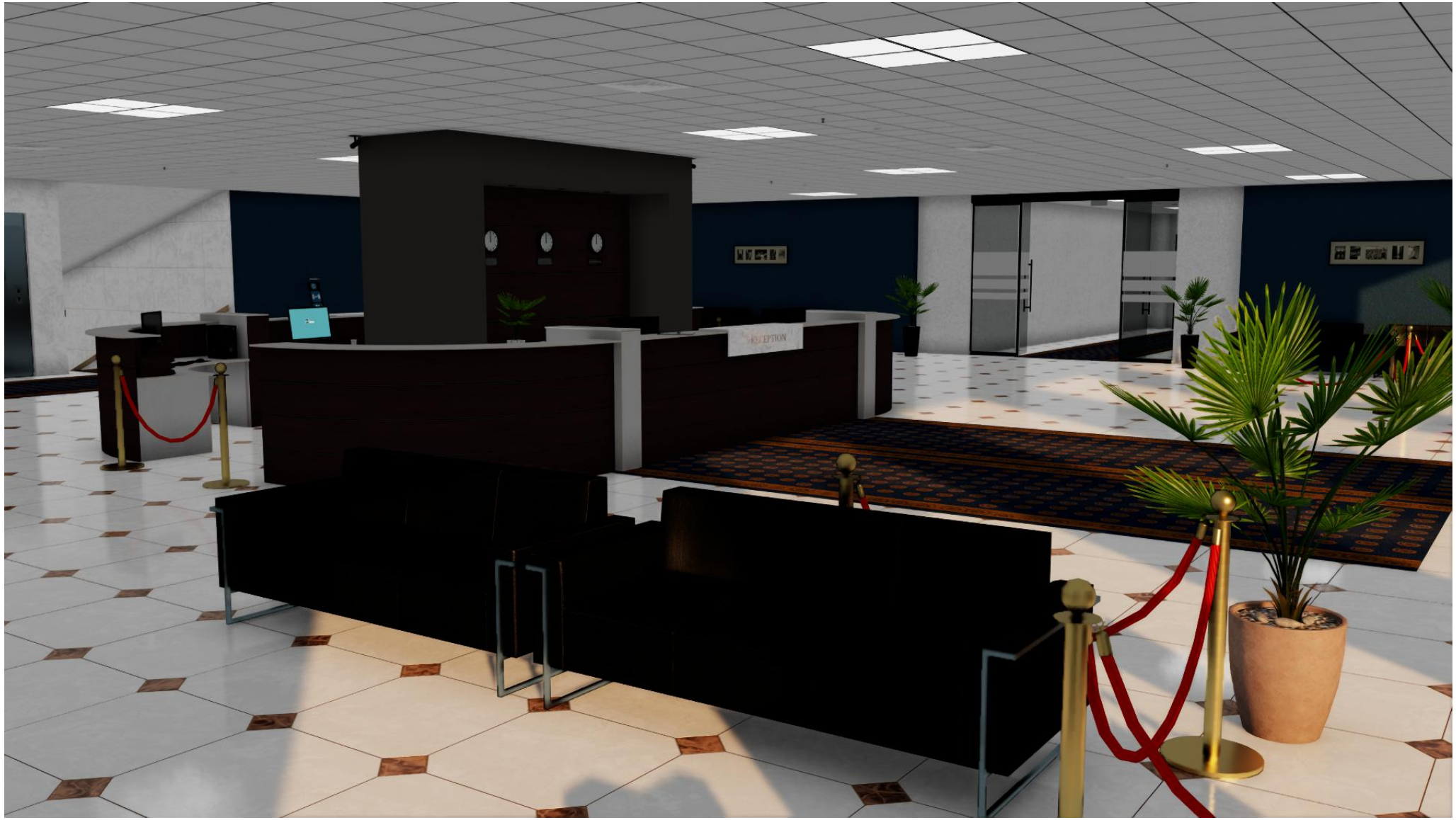
- **Isaac Gym** (legacy, first version was open source): pure GPU-accelerated physics for massively parallel RL; minimal visuals; now superseded by Lab for most research.

- **Isaac Lab** (proprietary core, open APIs): unifies Sim fidelity and Gym throughput; vectorized environments, actuator dynamics, procedural worlds, and clean APIs for RL/IL (RL-Games, Stable Baselines, custom PyTorch).

Everything visible is automatable and customizable via API but the core engine (renderer, physics solvers, etc.) is proprietary and distributed as pre-compiled binaries.

# Isaac Sim – Photoreal rendering with PhysX 5 engine

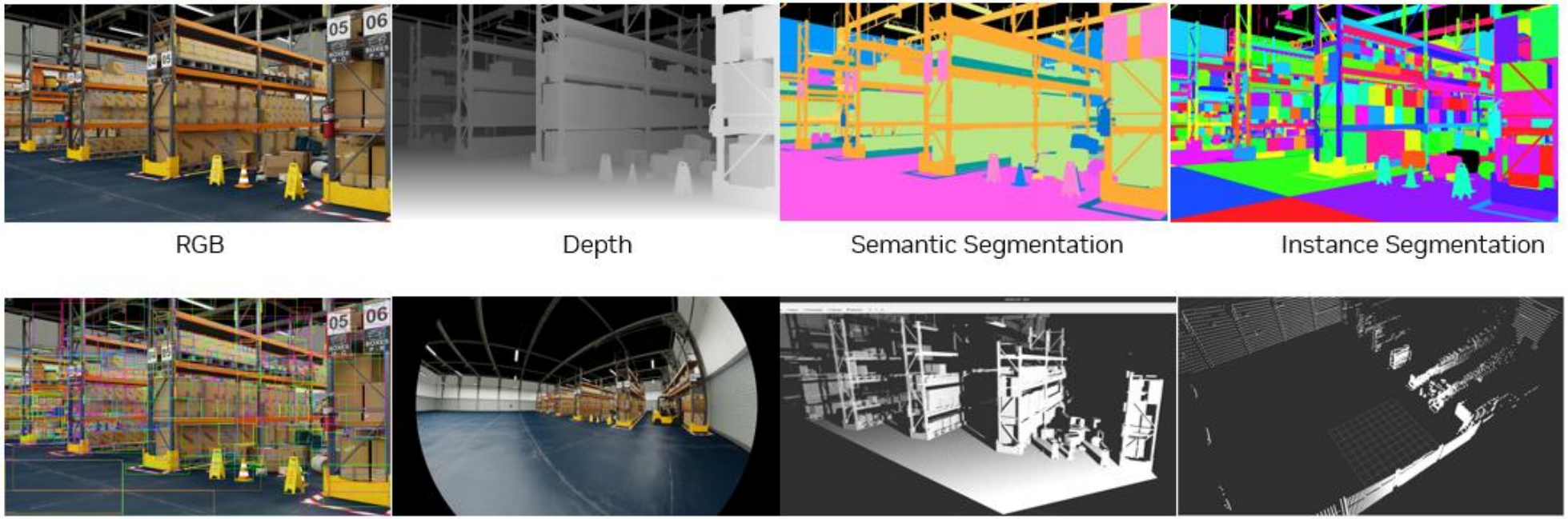
# Isaac Sim – Photoreal rendering with PhysX 5 engine



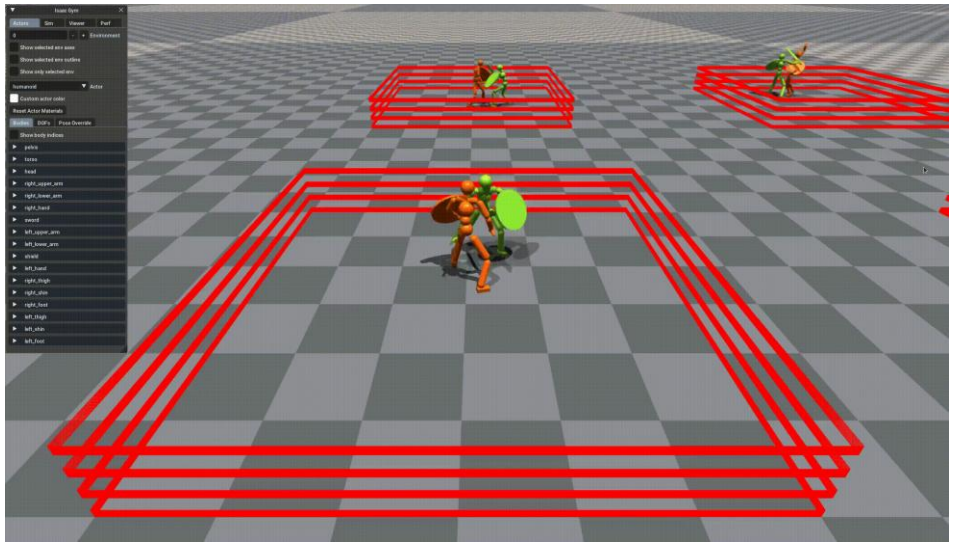
# Isaac Sim – Photoreal rendering with PhysX 5 engine

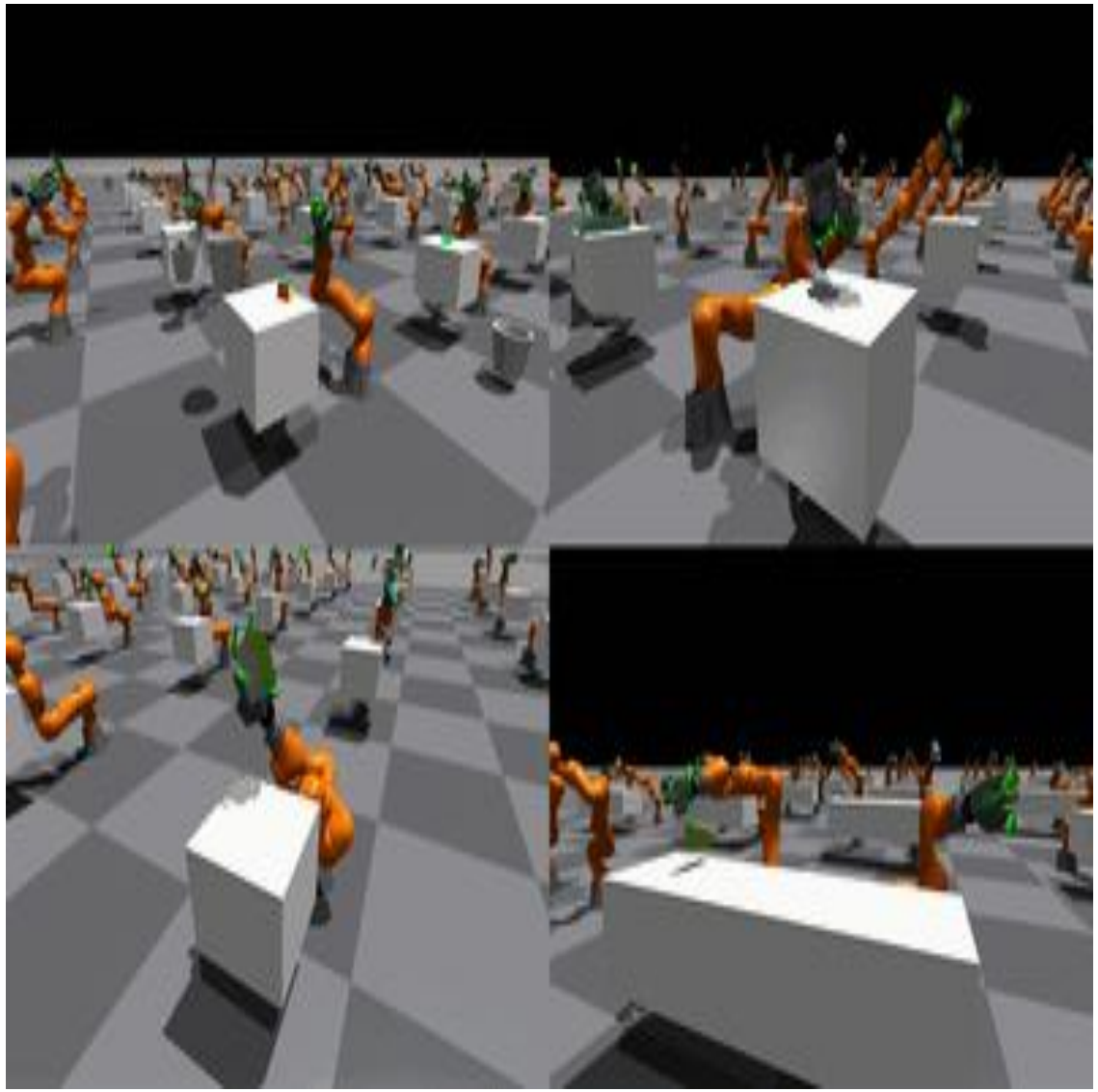
# Isaac Sim – Photoreal rendering with PhysX 5 engine

# Isaac Sim – Rich sensors (RGB-D, LiDAR, IMU)



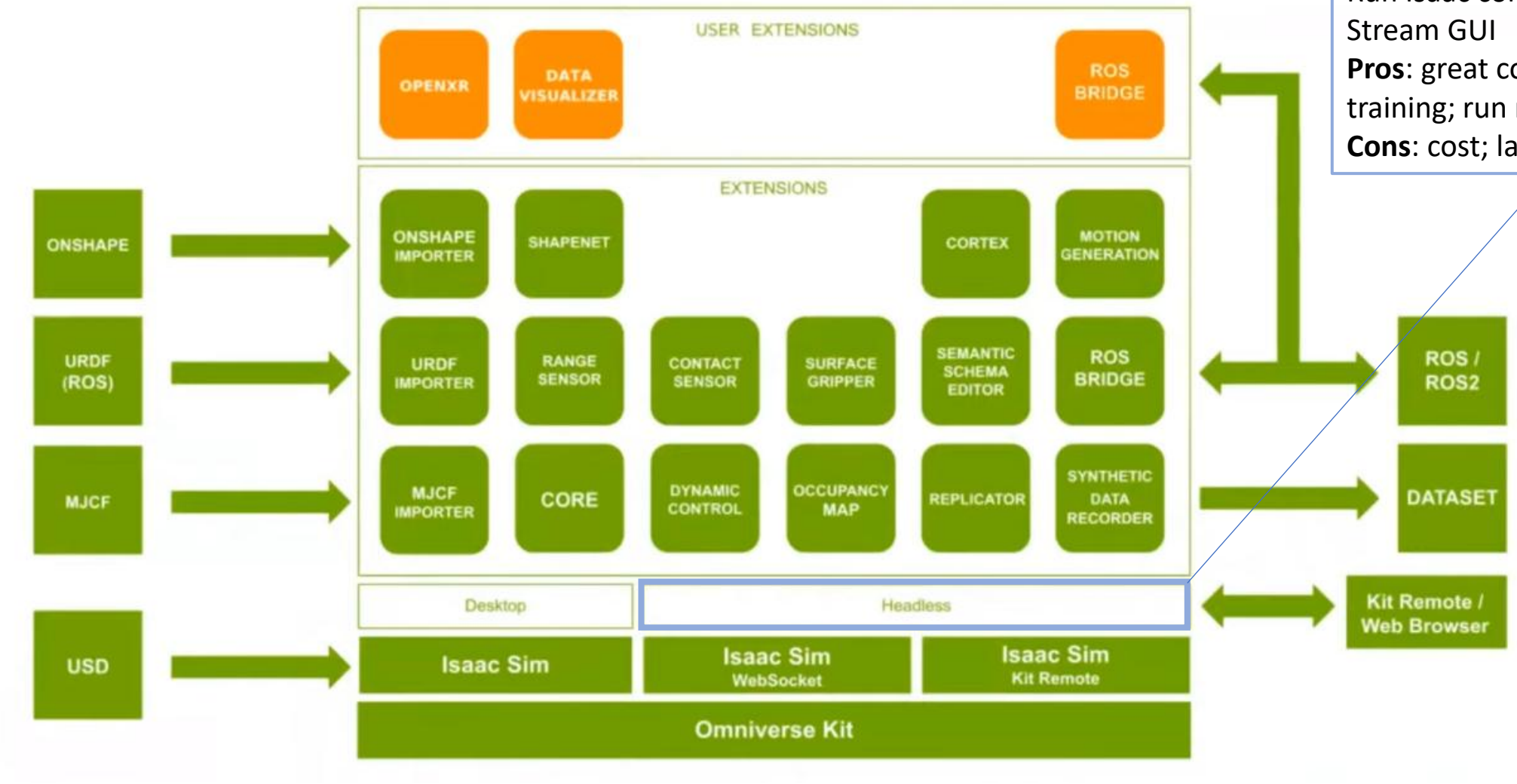
# Isaac Gym





# Isaac Lab

# Isaac Sim – System architecture




Run Isaac container headless
Stream GUI
**Pros**: great compute for training; run many parallel jobs
**Cons**: cost; latency

# Isaac Sim – Technical requirements

| Component | Minimum | Recommended |
|---|---|---|
| **OS** | Ubuntu 20.04/22.04 or Windows 10/11 (64-bit) | Ubuntu 22.04 LTS |
| **GPU** | NVIDIA RTX 3070 (≥8 GB VRAM) | RTX 4080 (≥16 GB) / RTX 6000 Ada (48 GB) |
| **CPU** | Modern 4+ core x86_64 | 8–16 cores |
| **RAM** | 32 GB | 64 GB+ |
| **Storage** | 50 GB free | 500 GB+ SSD |
| **Drivers** | Current NVIDIA driver | Latest studio/production driver |
| **Python** | Version pinned to release | Use Isaac-provided container |
| **Network** | Broadband | High-bandwidth, low-latency |

# Isaac vs non-specialized competition

| Platform | Pros | Cons |
|---|---|---|
| **NVIDIA Isaac** | Photorealism with PhysX; rich robotics (sensors, ROS2, USD assets); synthetic data generator & RL workflows; GPU; open-source | High GPU/VRAM demand; steeper learning curve; heavier than lightweight engines; |
| **MuJoCo** | Fast/accurate contact dynamics; great for complex articulated robots; open-source | Minimal graphics; fewer turnkey integrations (wraps needed); limited default parallelism (CPU) |
| **PyBullet** | Open-source, Python-friendly; easy URDF/MJCF import; great for quick prototyping | Lower contact fidelity; slower at scale (CPU); basic rendering; fragmented docs |
| **Unity ML-Agents** | User-friendly world building; good visuals; built-in multi-agent patterns; huge community | Game-oriented physics; limited large-scale parallel sim; C#/scene expertise; no native ROS/URDF |

# Toy robotic cell example

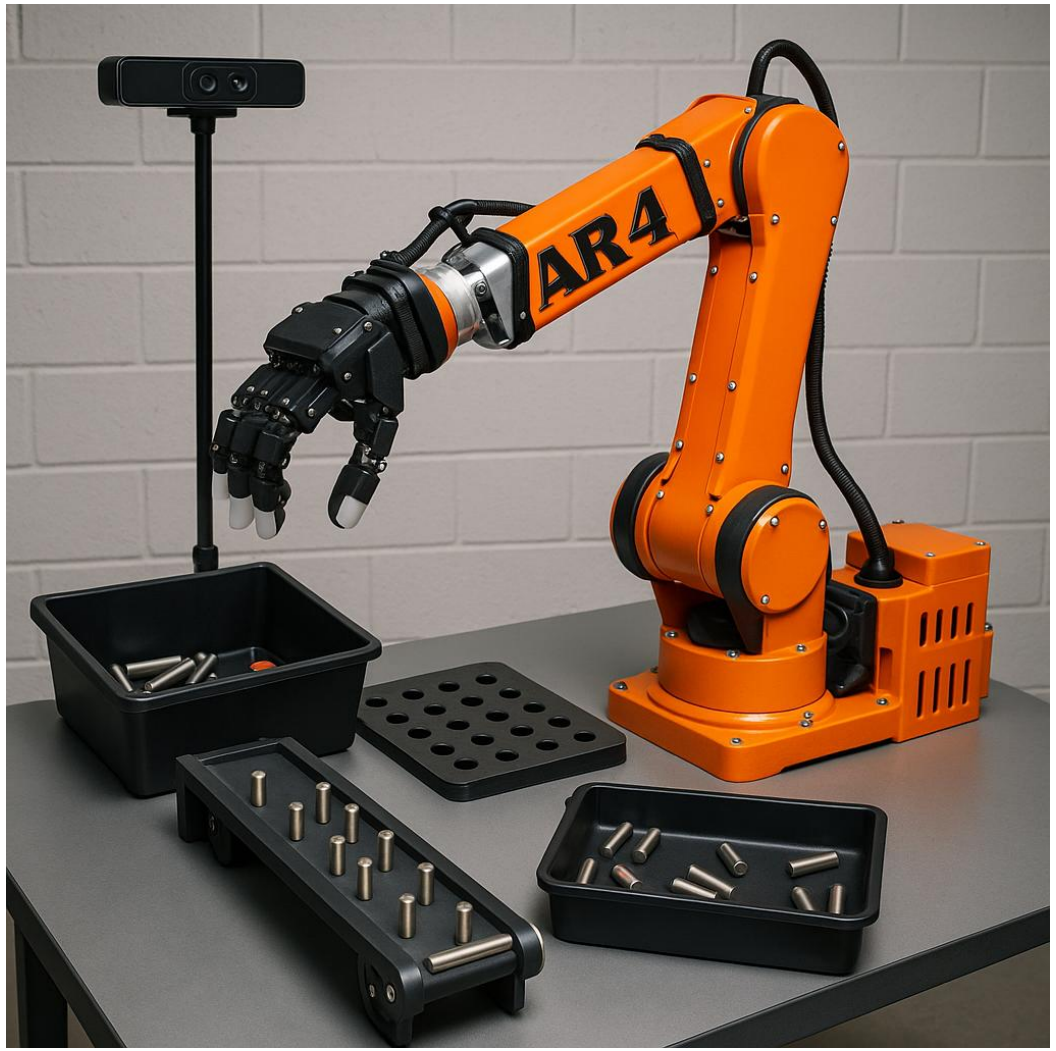
**Purpose**: Quality-controlled manipulation and kitting (representative of an assembly line)

**Physical setup:**

- 7-DoF arm and dexterous hand on a bench
- Sensors: arm and hand joints, F/T, haptic feedback, overhead RGB-D

**Mirror in Isaac Sim:**

- Import robots: URDF for arm and hand; verify joint limits, inertias, damping
- Create the scene: bench, bins, tray, conveyor; add PhysX materials (friction), collision shapes
- Add sensors to the scene
- Add ROS2 topics and controllers



# Advantages of Isaac for robotic applications

**Photorealism and Physics**

- RTX renderer: photoreal images for vision training; accurate shadows/reflections for realism
- PhysX 5 articulations and contacts: better grasp stability
- Hard/soft bodies and materials: more realistic interaction with objects
- Sensor realism: depth noise, lens distortion, LiDAR; for more robust perception models and better sim-to-real transfer

**Massively parallel learning** with Isaac Lab using vectorized environments on GPU

**Built-in synthetic data generation** (Replicator)

- Auto-labeling for dataset creation
- Domain randomization: materials, lighting, poses
- Accident and defect simulations

**Infrastructure stack friendly, easy to plug in big-data pipelines**

- Containers on NGC reproducible on Kubernetes/Kubeflow
- Experiment tracking with MLflow (rewards, losses, model registry)
- Streaming I/O: ROS2 bridge; simple kit extension for **Kafka** (MSK)

# Limitations of Isaac Sim

**Scope limitation**

- No meteorology engine: cannot simulate complex weather systems
- No PV/wind energy physics: does not compute irradiance-to-DC/AC-power or aero-servo-elastic turbine dynamics.
- No power-flow/dispatch: no AC power flow/OPF, market clearing, or feeder protection logic.
- No traffic flow model: no car-following, lane-changing, route choice, or network assignment.

**What that implies**

- Use **domain simulators** for authoritative physics:
  - PV: pvlib, NREL SAM
  - Wind: OpenFAST (FAST.Farm), windpowerlib
  - Grid: OpenDSS/PyDSS, GridLAB-D
  - Traffic: SUMO, MATSim, CityFlow

- Treat Isaac Sim as a 3D front-end and parallelizable **scenario generator**, **not as the solver** for those domains.

# Example of PV and wind in Isaac Sim as a scenario generator

**Step 1: Assemble the scene**

- PV: place USD assets for modules/strings/arrays, ground plane/terrain
- Wind: place turbine assets; enable simple rotation animation or articulation; add surrounding terrain/occluders.

**Step 2: generate sun and wind conditions**

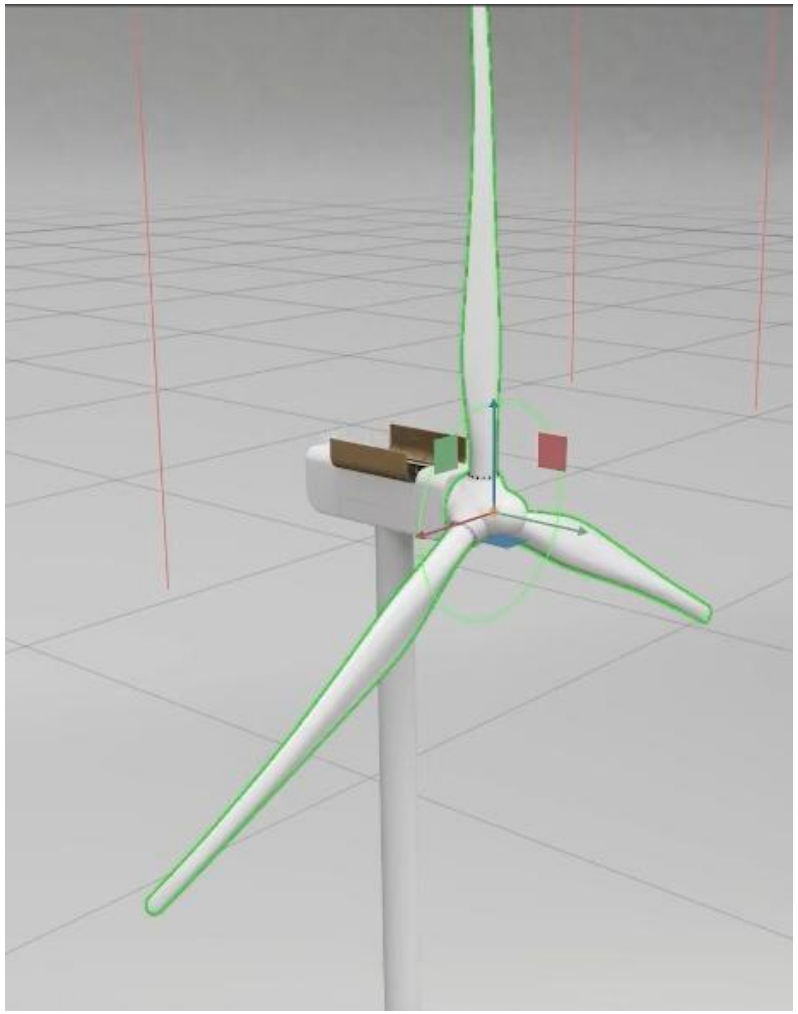
- Sun / lighting: animate a directional light (time-of-day / day-of-year).
- Add occluders (trees, buildings) to create partial shading patterns
- Wind exposure: use an external wind time series (reanalysis, forecast, sensors) or simple param scripts (speed/direction) to update a wind field and drive wind turbine rotor RPM for realism

**Step 3: Extract features running multiple scenarios** (the advantage of using Isaac)

- PV shading features (sampling light on panel surface)
- Wind exposure features (per turbine)

**Step 4: Hand off to specialized libraries (authoritative physics)**

- PV example: pvlib consumes sun + shading + weather and produces POA irradiance, predicted power output
- Wind example: OpenFAST consumes wind speed + direction + turbulence intensity and produces predicted rotational speed of the rotor and power output



# Conclusions

Isaac Sim **shines** with robotics fidelity, sensors, photorealistic rendering, and massively parallel vectorized environment for RL / synthetic dataset generation.

For energy and traffic, it **pairs** with the right specialized engines to speed up **counterfactuals exploration** without reinventing physics.

**Isaac Sim is not used for** building a weather, wind or PV model, but for **generating realistic, controllable conditions** (occluding shadows, layouts, force fields on rotors), potentially in a massively parallel way.

The payoff is richer **feature engineering** and **counterfactuals** for forecasting

It outline the importance of a **common data and technology stack** working with potentially **different** specialized plug-in/connected **simulators**